Подробный разбор модулей генератора

RealTime-AnalyticsPlatform · Модуль генерации данных

Разбор файлов последнего коммита feat(generator): metrics.py, catalog.py, weather.py, retry.py.

> Это дополнение к уже разобранным models.py, config.py, seasonality.py, anomalies.py. Здесь мы двигаемся «вверх» по слоям: от чистой математики сезонности — к генерации конкретных показаний, и добавляем устойчивость к отказам (retry).

---

1. metrics.py — самплеры метрик (паттерн Strategy)

Заголовок и импорты

```
from __future__ import annotations
```

Ленивые аннотации (PEP 563): все аннотации в файле — строки, не вычисляются при импорте. Здесь критично: MetricSampler.sample аннотирован через ReadingContext, объявленный ниже по файлу. Без __future__ был бы NameError.

```
import random
from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime
from .seasonality import annual_factor, diurnal_factor
```

- random — для распределений gauss, lognormvariate, uniform.
- abc (ABC, abstractmethod) — для абстрактного базового класса.
- dataclasses — для @dataclass.
- datetime — тип поля at в ReadingContext.
- .seasonality — относительный импорт «соседнего» модуля (из той же папки generation/).

ClimateProfile — климатический «фон» станции

```
@dataclass(frozen=True, slots=True)
class ClimateProfile:
  mean_annual_temp_c: float
  annual_temp_amplitude_c: float
  daily_temp_amplitude_c: float
  humidity_base_pct: float
  pressure_base_hpa: float = 1013.25
  wind_base_ms: float = 4.0
```

- @dataclass(frozen=True, slots=True) — общие кирпичики: генерирует конструктор/__eq__; frozen = неизменяемость; slots = компактное хранение.
- Это настройки города: средняя годовая температура, амплитуды (годовая/суточная), базовая влажность, давление, скорость ветра.
- У двух последних полей есть дефолты (1013.25 гПа — стандартное атмосферное давление на уровне моря, 4.0 м/с) — поэтому при создании профиля можно их не указывать.

> Зачем отдельный класс: он «параметризует» самплеры. Один и тот же код генерации работает для любого города — достаточно подставить свой ClimateProfile. Это суть паттерна Strategy: алгоритм одинаковый, а

> поведение меняется параметрами.

ReadingContext — общий контекст показания

```
@dataclass(frozen=True, slots=True)
class ReadingContext:
  at: datetime
  temperature_c: float
```

Контекст, который самплеры получают в sample(ctx, rng). Содержит: - at — момент времени; - temperature_c — уже посчитанную температуру.

Зачем temperature_c: чтобы влажность/давление могли коррелировать с температурой (реалистичная динамика). HumiditySampler использует ctx.temperature_c — влажность падает при повышении температуры.

> Обратите внимание: ReadingContext создаётся дважды в weather.py — сначала с temperature_c=0.0 (для расчёта температуры), потом заново с реальной температурой (для остальных самплеров). Так как это frozen dataclass, его нельзя изменить — проще создать новый.

_clamp — вспомогательная функция

```
def _clamp(value: float, low: float, high: float) -> float:
  return max(low, min(high, value))
```

Приватная функция (префикс _), «зажимает» значение в диапазон [low, high]: - min(high, value) — если value > high, берём high; - max(low, ...) — если получилось меньше low, берём low.

Зачем: физические величины ограничены (влажность не бывает 120%, давление — 500 гПа). Самплеры применяют _clamp к результату, чтобы не выдать абсурдное значение.

MetricSampler — абстрактный контракт (Strategy)

```
class MetricSampler(ABC):
  @abstractmethod
  def sample(self, ctx: ReadingContext, rng: random.Random) -> float: ...
```

- ABC — абстрактный базовый класс: нельзя создать экземпляр MetricSampler().
- @abstractmethod на sample — метод «абстрактный»: подклассы обязаны его реализовать. Если не реализовать — класс тоже станет абстрактным и не инстанцируется.
- Тело ... (Ellipsis) — только сигнатура, реализации нет.
- Контракт: sample(ctx, rng) -> float — принимает контекст и генератор, возвращает число.

> Это сердце паттерна Strategy: единый интерфейс + разные реализации. Чтобы добавить новую метрику (например, уровень солнечной радиации) — пишем один класс RadiationSampler(MetricSampler) и реализуем sample. Остальной код (WeatherReadingGenerator) не меняется — принцип открытости/закрытости (OCP).

TemperatureSampler — температура

```
class TemperatureSampler(MetricSampler):
  def __init__(self, profile: ClimateProfile, noise_std: float = 1.2) -> None:
    self._profile = profile
```

```
        self._noise_std = noise_std

    def sample(self, ctx: ReadingContext, rng: random.Random) -> float:
        base = (
            self._profile.mean_annual_temp_c
            + self._profile.annual_temp_amplitude_c * annual_factor(ctx.at)
            + self._profile.daily_temp_amplitude_c * diurnal_factor(ctx.at)
        )
        return round(base + rng.gauss(0.0, self._noise_std), 2)
```

Формула (из разобранного seasonality.py):

```
температура = средняя годовая + годовая амплитуда × годовой множитель
            + суточная амплитуда × суточный множитель + шум
```

По частям: - mean_annual_temp_c — базовый климат города (например, для Москвы 6.0°C). - annual_factor(ctx.at) — сезонная поправка [-1, 1]: летом +1 (прибавляет annual_temp_amplitude_c), зимой −1. - diurnal_factor(ctx.at) — суточная поправка [-1, 1]: в 15:00 +1, в 03:00 −1. - rng.gauss(0.0, self._noise_std) — нормально распределённый шум (гауссов) с мат. ожиданием 0 и стандартным отклонением noise_std = 1.2. Делает значения «живыми», а не идеально гладкими. - round(..., 2) — округляем до 2 знаков (реалистичная точность метеоданных).

> rng.gauss использует сидированный генератор — поэтому шум детерминирован при фиксированном сиде.

HumiditySampler — влажность (корреляция с температурой)

```
class HumiditySampler(MetricSampler):
    def __init__(self, profile: ClimateProfile) -> None:
        self._profile = profile

    def sample(self, ctx: ReadingContext, rng: random.Random) -> float:
        deviation = ctx.temperature_c - self._profile.mean_annual_temp_c
        value = self._profile.humidity_base_pct - 0.8 * deviation + rng.gauss(0.0, 3.0)
        return round(_clamp(value, 5.0, 100.0), 2)
```

- deviation — насколько текущая температура отклонилась от годовой нормы города.
- 0.8 * deviation — коэффициент корреляции −0.8: чем выше температура (положительное отклонение), тем ниже влажность. Реалистично: в жару влажность падает.
- rng.gauss(0.0, 3.0) — шум (разброс 3 пункта процента).
- _clamp(value, 5.0, 100.0) — влажность физически от 5% до 100%.

PressureSampler — давление

```
class PressureSampler(MetricSampler):
    def __init__(self, profile: ClimateProfile, tide_amplitude: float = 1.2) -> None:
        self._profile = profile
        self._tide_amplitude = tide_amplitude

    def sample(self, ctx: ReadingContext, rng: random.Random) -> float:
        tide = self._tide_amplitude * diurnal_factor(ctx.at)
        value = self._profile.pressure_base_hpa + tide + rng.gauss(0.0, 0.8)
        return round(_clamp(value, 850.0, 1100.0), 2)
```

- tide — слабый суточный «прилив» давления (амплитуда 1.2 гПа), используется diurnal_factor. Реальный физический эффект — атмосферное давление слегка колеблется в течение суток.
- pressure_base_hpa — базовое давление (по умолчанию 1013.25 гПа).
- rng.gauss(0.0, 0.8) — небольшой шум.

- _clamp(value, 850.0, 1100.0) — разумный физический диапазон давления (850–1100 гПа).

WindSpeedSampler — скорость ветра

```
class WindSpeedSampler(MetricSampler):
  def __init__(self, profile: ClimateProfile) -> None:
    self._profile = profile

  def sample(self, ctx: ReadingContext, rng: random.Random) -> float:
    value = self._profile.wind_base_ms * rng.lognormvariate(0.0, 0.55)
    return round(_clamp(value, 0.0, 60.0), 2)
```

- rng.lognormvariate(0.0, 0.55) — логнормальное распределение. В отличие от гауссова (симметричного), логнормальное асимметрично — даёт много «тихих» значений и редкие «порывы».
- Умножается на wind_base_ms — базовую скорость города.
- _clamp(value, 0.0, 60.0) — скорость не бывает отрицательной; верхняя граница 60 м/с (ураган) как защита.

> Зачем логнормальное, а не нормальное: реальный ветер «чаще тихий, реже сильный». Нормальное распределение дало бы симметричный шум вокруг среднего, что физически неверно.

WindDirectionSampler — направление ветра

```
class WindDirectionSampler(MetricSampler):
  def __init__(self, profile: ClimateProfile, step_deg: float = 15.0) -> None:
    self._profile = profile
    self._step_deg = step_deg

  def sample(self, ctx: ReadingContext, rng: random.Random) -> float:
    value = rng.uniform(0.0, 360.0)
    return round(value, 1)
```

- rng.uniform(0.0, 360.0) — равномерное случайное направление от 0° до 360°.
- _step_deg — параметр, зарезервированный для будущего «медленного блуждания» направления (чтобы направление не прыгало скачками между соседними показаниями). Пока не используется, но заложено в контракт.
- round(value, 1) — округление до 1 знака.

---

2. catalog.py — справочник станций

Импорты

```
from __future__ import annotations
from dataclasses import dataclass
from generator.models import Station
from .metrics import ClimateProfile
```

- dataclasses — для @dataclass.
- generator.models.Station — доменная модель станции.
- .metrics.ClimateProfile — климатический профиль (относительный импорт).

_GENERIC — запасной профиль

```
_GENERIC = ClimateProfile(
  mean_annual_temp_c=10.0,
  annual_temp_amplitude_c=20.0,
  daily_temp_amplitude_c=7.0,
  humidity_base_pct=70.0,
)
```

Профиль «по умолчанию» для неизвестных городов. Создаётся один раз на уровне модуля. Зачем: система не должна падать, если пользователь укажет в конфиге город, которого нет в справочнике — для него будет «средний климат».

_Template — шаблон города

```
@dataclass(frozen=True, slots=True)
class _Template:
  city: str
  latitude: float
  longitude: float
  altitude: float
  climate: ClimateProfile
```

Приватная (_) неизменяемая структура: город + координаты + климат. Это «сырые» данные, из которых потом строятся Station.

_TEMPLATES — словарь справочника

```
_TEMPLATES: dict[str, _Template] = {
  "Vladivostok": _Template("Vladivostok", 43.12, 131.89, 40, ClimateProfile(...)),
  "Moscow": _Template("Moscow", 55.76, 37.62, 156, ClimateProfile(...)),
  "Kazan": ..., "Sochi": ..., "Irkutsk": ...,
}
```

- dict[str, _Template] — словарь: ключ — название города (строка), значение — его шаблон.
- Аннотация dict[str, _Template] типизирует и ключ, и значение.
- Это данные по умолчанию для пяти городов. У каждого свой климат (например, Иркутск — континентальный, с огромной годовой амплитудой 33°C; Сочи — субтропический, амплитуда всего 14°C).

StationCatalog — фабрика станций

```
class StationCatalog:
  def __init__(self, cities: list[str]) -> None:
    self._templates = [self._resolve(city) for city in cities]

  @staticmethod
  def _resolve(city: str) -> _Template:
    return _TEMPLATES.get(city, _Template(city, 0.0, 0.0, 100.0, _GENERIC))
```

- __init__ принимает список городов (из конфига city_list). Для каждого города находит шаблон и кладёт в self._templates.
- @staticmethod _resolve — декоратор: метод не получает self/cls, ведёт себя как обычная функция, но живёт в классе.
- _TEMPLATES.get(city, fallback) — dict.get возвращает значение по ключу, а если ключа нет — возвращает fallback. Здесь fallback — город с нулевыми координатами и обобщённым климатом. Так неизвестный город не роняет систему.

```
    def stations(self) -> list[Station]:
        return [
            Station(
                station_id=index,
                name=f"{t.city} Station",
                city=t.city,
                latitude=t.latitude,
                longitude=t.longitude,
                altitude=t.altitude,
            )
            for index, t in enumerate(self._templates)
        ]
```

- stations() — list comprehension (списковое включение): строит список Station из шаблонов.
- enumerate(self._templates) — возвращает пары (индекс, элемент). Индекс становится station_id (0, 1, 2, …).
- name=f"{t.city} Station" — f-строка: имя «Vladivostok Station».
- Каждый вызов stations() создаёт новые объекты Station — это фабрика.

```
    def climate_for(self, station: Station) -> ClimateProfile:
        return next(t.climate for t in self._templates if t.city == station.city)
```

- climate_for — находит климат по станции.
- next(generator_expression) — берёт первый элемент из генераторного выражения. Генераторное выражение (t.climate for t in ... if ...) — «ленивое»: перебирает шаблоны, пока не найдёт совпадение по городу, и next() забирает первый результат.

> Здесь «ленивость» — в генераторном выражении: оно вычисляется не целиком, а по мере запроса, и next() останавливает его на первом совпадении (не перебирая весь список зря).

---

3. weather.py — генератор показаний

Импорты

```
import random
from datetime import datetime
from generator.models import Station, WeatherReading, WeatherType
from .anomalies import AnomalyKind, AnomalyPolicy
from .catalog import StationCatalog
from .metrics import (ClimateProfile, HumiditySampler, MetricSampler,
                      PressureSampler, ReadingContext, TemperatureSampler,
                      WindDirectionSampler, WindSpeedSampler)
```

- random — random.Random(seed).
- datetime — тип времени.
- Модели: Station, WeatherReading, WeatherType.
- Аномалии: AnomalyKind, AnomalyPolicy.
- Каталог: StationCatalog.
- Все самплеры из metrics.

_WeatherTypePicker — выбор категории погоды

```
class _WeatherTypePicker:
    def pick(self, ctx, humidity, pressure, wind, anomaly, rng) -> WeatherType:
        if anomaly == AnomalyKind.WIND_GUST or wind > 20.0:
            return WeatherType.STORM
```

```
        if anomaly == AnomalyKind.PRESSURE_DROP or pressure < 1000.0:
            return WeatherType.RAIN
        if humidity > 75.0 and rng.random() < 0.35:
            return WeatherType.SNOW if ctx.temperature_c <= 0.0 else WeatherType.RAIN
        if humidity > 80.0 and wind < 1.5:
            return WeatherType.FOG
        if humidity < 60.0:
            return WeatherType.CLEAR
        return WeatherType.CLOUDY
```

Приватный (_) вспомогательный класс. pick — детерминированная эвристика (правила), выбирающая категорию погоды по показаниям и аномалии. Правила по порядку:

1. Аномалия порыва ветра или ветер > 20 м/с → STORM (буря).
2. Аномалия падения давления или давление < 1000 гПа → RAIN (дождь).
3. Влажность > 75% и rng.random() < 0.35 → случайно SNOW (если ≤0°C) или RAIN. Здесь rng нужен, чтобы «дождь/снег» не были детерминированными.
4. Влажность > 80% и слабый ветер (< 1.5 м/с) → FOG (туман).
5. Влажность < 60% → CLEAR (ясно).
6. Иначе → CLOUDY (облачно).

Порядок важен: проверки идут от «экстремальных» к «обычным» — буря имеет приоритет над облачностью.

WeatherReadingGenerator — фасад генерации

```
class WeatherReadingGenerator:
    def __init__(self, station, profile, anomaly_policy, rng):
        self._station = station
        self._rng = rng
        self._anomaly_policy = anomaly_policy
        self._temp = TemperatureSampler(profile)
        self._humidity = HumiditySampler(profile)
        self._pressure = PressureSampler(profile)
        self._wind = WindSpeedSampler(profile)
        self._direction = WindDirectionSampler(profile)
        self._weather_type = _WeatherTypePicker()
```

Инъекция зависимостей: генератор получает станцию, профиль, политику аномалий и rng извне — не создаёт их сам. Зачем: тестируемость и подстановка.

_temp, _humidity и т.д. — экземпляры самплеров (Strategy). Здесь видно, как фасад компонует стратегии.

```
    @property
    def station(self) -> Station:
        return self._station
```

@property — read-only доступ к станции.

```
    def generate(self, at: datetime) -> WeatherReading:
        anomaly = self._anomaly_policy.decide()
        rng = self._rng

        ctx = ReadingContext(at=at, temperature_c=0.0)
        temperature = self._temp.sample(ctx, rng)
        temperature = self._apply_temperature_anomaly(temperature, anomaly)

        ctx = ReadingContext(at=at, temperature_c=temperature)
        humidity = self._humidity.sample(ctx, rng)
        pressure = self._apply_pressure_anomaly(self._pressure.sample(ctx, rng), anomaly)
```

```
    wind = self._apply_wind_anomaly(self._wind.sample(ctx, rng), anomaly)
    direction = self._direction.sample(ctx, rng)

    weather_type = self._weather_type.pick(ctx, humidity, pressure, wind, anomaly, rng)

    return WeatherReading(
        station_id=self._station.station_id,
        timestamp=at,
        temperature_c=temperature,
        humidity_pct=humidity,
        pressure_hpa=pressure,
        wind_speed_ms=wind,
        wind_direction_deg=direction,
        weather_type=weather_type,
        is_anomaly=anomaly is not None,
    )
```

Это главный метод. По шагам:

1. anomaly = self._anomaly_policy.decide() — решаем, будет ли аномалия (разобрано в anomalies.py).
2. ctx = ReadingContext(at=at, temperature_c=0.0) — контекст с заглушкой температуры (0.0), потому что температуру ещё не посчитали.
3. temperature = self._temp.sample(ctx, rng) — температура.
4. temperature = self._apply_temperature_anomaly(temperature, anomaly) — если выпала аномалия тепла/холода, применяем её.
5. Создаём новый ReadingContext уже с реальной температурой — для влажности/давления (корреляция).
6. humidity, pressure, wind, direction — остальные самплеры; к давлению и ветру тоже применяются аномалии.
7. weather_type — категория погоды.
8. Собираем WeatherReading (неизменяемый dataclass) — is_anomaly=anomaly is not None — флаг для анализа.

> Обратите внимание на порядок вызовов rng: он строго детерминирован — сначала decide(), потом температура, потом влажность и т.д. Именно этот стабильный порядок + сид и дают воспроизводимость.

Применение аномалий

```
  def _apply_temperature_anomaly(self, value, anomaly):
      strength = self._anomaly_policy.strength
      if anomaly == AnomalyKind.HEAT_WAVE:
          return round(value + strength * self._rng.uniform(5.0, 8.0), 2)
      if anomaly == AnomalyKind.COLD_SNAP:
          return round(value - strength * self._rng.uniform(5.0, 8.0), 2)
      return value

  def _apply_pressure_anomaly(self, value, anomaly):
      if anomaly == AnomalyKind.PRESSURE_DROP:
          drop = self._anomaly_policy.strength * self._rng.uniform(8.0, 25.0)
          return round(max(850.0, value - drop), 2)
      return value

  def _apply_wind_anomaly(self, value, anomaly):
      if anomaly == AnomalyKind.WIND_GUST:
          gust = value * self._anomaly_policy.strength * self._rng.uniform(2.0, 3.5)
          return round(min(60.0, gust), 2)
      return value
```

- _apply_temperature_anomaly: для HEAT_WAVE прибавляет strength × uniform(5, 8); для COLD_SNAP — вычитает. strength — сила (множитель из конфига, по умолчанию 3.0). Если аномалия не относится к температуре — возвращает значение без изменений (return value).
- _apply_pressure_anomaly: для PRESSURE_DROP вычитает strength × uniform(8, 25) гПа, но max(850.0, ...) не даёт давлению упасть ниже 850 гПа.

- _apply_wind_anomaly: для WIND_GUST умножает на strength × uniform(2.0, 3.5), но min(60.0, ...) не даёт превысить 60 м/с.

Ключевая идея: decide() (в anomalies.py) решает факт и тип аномалии, а здесь — величину (силу скачка). Разделение ответственности.

BatchGenerator — генерация пачки показаний

```
class BatchGenerator:
  def __init__(self, catalog: StationCatalog, seed: int, anomaly_policy: AnomalyPolicy) -> None:
    rng = random.Random(seed)
    self._per_station = [
      WeatherReadingGenerator(s, catalog.climate_for(s), anomaly_policy, rng)
      for s in catalog.stations()
    ]
```

- rng = random.Random(seed) — один сидированный генератор для всех станций.
- Важно: один rng передаётся всем генераторам станций. Значит, все они «тянут» числа из одного общего потока. Это даёт детерминизм, но и подводный камень: изменение числа вызовов в одном месте сдвигает весь поток.
- catalog.climate_for(s) — профиль для каждой станции.
- _per_station — список генераторов, по одному на станцию.

```
  @property
  def stations(self) -> list[Station]:
    return [g.station for g in self._per_station]
```

@property stations — список станций через read-only station каждого генератора.

```
  def generate_batch(self, at: datetime, size: int) -> list[WeatherReading]:
    if not self._per_station:
      return []
    readings: list[WeatherReading] = []
    for index in range(size):
      generator = self._per_station[index % len(self._per_station)]
      readings.append(generator.generate(at))
    return readings
```

- if not self._per_station: return [] — защита: если станций нет, вернуть пустой список.
- index % len(self._per_station) — оператор взятия остатка от деления %. Он «зацикливает» индекс: при index = 0,1,2,... и, скажем, 5 станциях, даёт 0,1,2,3,4,0,1,2,.... Так записи равномерно чередуются между станциями в батче.
- readings: list[WeatherReading] = [] — аннотация переменной, помогает mypy.
- В цикле генерируем size показаний, распределяя по станциям.

---

4. retry.py — устойчивость к отказам

Импорты

```
import logging
import random
import time
from collections.abc import Callable
from typing import TypeVar
```

- logging — логгер для сообщений о повторах.
- random — джиттер паузы.
- time — time.sleep для реальной паузы.
- collections.abc.Callable — тип «функция».
- typing.TypeVar — обобщённый тип.

T = TypeVar("T")

```
T = TypeVar("T")
```

TypeVar — объявляет обобщённый параметр типа. Зачем: метод execute может обернуть любую функцию, и мы хотим, чтобы тип возврата сохранялся.

```
def execute(self, fn: Callable[[], T]) -> T:
```

Если fn возвращает list[Station], то T = list[Station], и execute возвращает list[Station]. Без TypeVar пришлось бы возвращать Any и потерять проверку типов в местах вызова.

RetryError — своё исключение

```
class RetryError(Exception):
  """Все попытки исчерпаны."""
```

Пользовательское исключение — чтобы вызывающий код мог отличить «действие не удалось после N попыток» от других ошибок (в GeneratorService его ловят отдельно).

RetryPolicy — политика повторных попыток

```
class RetryPolicy:
  def __init__(self, max_attempts, base_delay, max_delay, backoff_factor=2.0,
               jitter=0.1, sleep=time.sleep):
    self._max_attempts = max_attempts
    self._base_delay = base_delay
    self._max_delay = max_delay
    self._backoff_factor = backoff_factor
    self._jitter = jitter
    self._sleep = sleep
```

Параметры: - max_attempts — сколько всего попыток. - base_delay — начальная пауза. - max_delay — потолок паузы. - backoff_factor — множитель роста паузы (2.0 = «задвоение»). - jitter — случайный разброс паузы. - sleep — инъекция функции сна (по умолчанию time.sleep). Зачем: в тестах подменяют на фиктивный sleep, чтобы не ждать реально.

---

execute — основной метод

```
  def execute(self, fn: Callable[[], T]) -> T:
    attempt = 0
    while True:
      try:
        return fn()
      except Exception as exc:
```

```
            attempt += 1
            if attempt >= self._max_attempts:
                raise RetryError(f"действие не удалось после {self._max_attempts} попыток") from exc
            delay = self._backoff_delay(attempt)
            logger.warning("Сбой попытки %d/%d (%s); повтор через %.2fs", attempt, self._max_attempts, exc, delay)
            self._sleep(delay)
```

Логика: 1. attempt = 0, while True: — бесконечный цикл с попытками. 2. try: return fn() — пробуем выполнить. Если успешно — сразу возвращаем результат (выходим из цикла). 3. except Exception as exc: — если упало: - attempt += 1 — увеличиваем счётчик. - if attempt >= max_attempts: raise RetryError(...) from exc — исчерпали попытки: бросаем RetryError, а from exc сохраняет исходную ошибку как контекст (для отладки). - delay = self._backoff_delay(attempt) — вычисляем паузу. - logger.warning(...) — ленивое логирование (см. ниже). - self._sleep(delay) — ждём (через инъекцию _sleep, а не напрямую time.sleep — тестируемость).

Ленивые строки в логгере:

```
logger.warning("Сбой попытки %d/%d (%s); повтор через %.2fs", attempt, self._max_attempts, exc, delay)
```

Здесь нет f-строки! Форматирование %d/%s/%.2f произойдёт только если warning реально выведется (уровень логирования позволяет). Это классический ленивый паттерн логирования — не тратим CPU на форматирование, когда сообщение отбрасывается.

_backoff_delay — формула экспоненциального backoff

```
    def _backoff_delay(self, attempt: int) -> float:
        base = self._base_delay * (self._backoff_factor ** (attempt - 1))
        capped = min(base, self._max_delay)
        return capped * (1.0 + random.uniform(-self._jitter, self._jitter))
```

Формула экспоненциального backoff с джиттером:

1. self._backoff_factor ** (attempt - 1) — возведение в степень. При backoff_factor = 2.0:
2. попытка 1 → 2^0 = 1,
3. попытка 2 → 2^1 = 2,
4. попытка 3 → 2^2 = 4 (пауза растёт вдвое каждый раз).
5. base = base_delay * ... — начальная пауза умножается на это.
6. capped = min(base, max_delay) — ограничение сверху: пауза не растёт бесконечно, упирается в max_delay.
7. capped * (1.0 + uniform(-jitter, jitter)) — джиттер: умножаем на случайный коэффициент от 0.9 до 1.1 (при jitter=0.1). Зачем: если все клиенты перестают и перезапускают запросы одновременно, они «собьются в стаю» и повторно нагрузят БД разом; джиттер «размазывает» паузы, чтобы запросы не совпадали.

Сводка по паттернам в этих четырёх файлах

Файл	Паттерн / принцип	Ключевой приём
metrics.py	Strategy	единый контракт sample(ctx, rng), разные распределения
catalog.py	Factory	StationCatalog строит станции из шаблонов
weather.py	Facade + Strategy	фасад компонует самплеры; % чередует станции

Файл	Паттерн / принцип	Ключевой приём
retry.py	обёртка (decorator-идея)	экспоненциальный backoff + джиттер, инъекция sleep

Все четыре — про инъекцию зависимостей (rng, sleep, профили передаются извне), детерминизм (один сидированный rng) и инкапсуляцию (приватные поля/методы _).